Introduction to AI and ML

Project Report

Career Suggestion Expert System using Prolog

Name:
Aarush Sudheer

Registration Number:
25BAI10711

Type:
Project Report

Institution:
VIT University, Bhopal

1. Introduction

Career decisions can be really tough. They seem like they should be easy. In reality most people I know including myself end up choosing a career based on what the people around us are doing and what our family thinks is a good idea. We do not really think about what we're interested in and what we are good at until later and sometimes that is after we have already made our choice.

This project started because of this problem. I wanted to build an expert system in Prolog that asks the user for some information like what they are interested in how good they are at something and what kind of work they like and then gives them some career suggestions based on what it knows and the rules it follows. This system is not meant to replace getting advice from someone who knows what they are talking about. I want to make that clear.. It is a working example of how we can use rules to make decisions that people usually make without thinking about it and building it was a good way to learn about expert systems.

2. Problem Statement

The main problem is that students often end up in careers that're not a good fit for them not because they are unlucky but because they did not have enough information or did not think about it carefully. There are a reasons why this happens:

* People are not always sure what they are interested in and they are not always good at figuring out what they really enjoy doing versus what they have been told is valuable

* It is hard to match skills and career requirements without some kind of framework especially when a student has no work experience

* The advice that students get from counselors, schools or career fairs is often too general and not tailored to the students needs

As a result choosing a career often becomes a process of trying things out or copying what others are doing rather than finding a career that is a good match. I thought it would be useful to build a tool that makes the matching process more explicit even if it's a simple one.

3. Objective

When I started this project I had three goals and one informal goal. My main goals were:

* to build a system that actually works and gives results rather than just being a design document

* to make it work by using logical rules rather than just looking up answers so the system can reason and come to conclusions

* to implement it in Prolog, which is the language we learned in this course and which is well-suited to this kind of problem

My informal goal was to understand what I built well enough to explain it to someone else. This is harder than it sounds. If I can run a system but cannot explain how it works then I do not really understand it. One of the things I wanted to get out of this project was an understanding of how Prolog works, not just a working file that I can submit.

4. Proposed Solution

The system asks the user for three pieces of information: their area of interest their level of skill and the type of work they prefer. It then uses this information to check against a knowledge base that stores career data as facts with each career tagged with its interest area, skill level and work type. The system uses rules to check whether a career matches the users input and if it does it returns that career as a suggestion.

What makes this an expert system, than just a list of careers is that it uses rules to draw conclusions rather than just looking up answers. A spreadsheet can return rows that match a query. A rule-based system evaluates conditions and infers which conclusions follow. For a project like this the difference may not be huge but the underlying mechanism is different and understanding that difference was one of my goals.

5. Tools and Technologies Used

I used three tools for this project, each with its role:

* SWI-Prolog. This is where I wrote and tested the program. SWI-Prolog is well-documented and widely used for this kind of work which made it easier to find help when I got stuck

* Visual Studio Code. I used this as my editor. It does not have any features for Prolog but the Prolog extension added syntax highlighting, which was helpful when I was trying to find errors in my code

* GitHub. I used this to keep track of my project as I worked on it. It turned out to be more useful than I expected because I could go back to a version of my project if I made a change that did not work

6. System Design

The design of the system is based on three ideas that describe what the user is looking for:

* Interest. The area that the user finds engaging with options like coding, design, business and research

* Skill level. The users assessment of their skills ranging from low to medium to high

* Work type. The kind of work the user prefers, either creative, analytical or practical

Each career in the knowledge base is associated with one value for each of these ideas. When the users input matches a careers attributes that career is returned as a suggestion. If multiple careers match all of them are returned. If none match the system tells the user than just returning nothing, which is an important design choice for usability.

I kept the design simple on purpose. A complex system with weighted scores or probabilistic matching would give more nuanced results but it would also be harder to understand. For this project I prioritized making the reasoning transparent over making the output sophisticated.

7. Features of the System

The system has a features that make it more usable:

* A step-by-step menu that walks the user through each input so they do not have to understand the interface from scratch

* Input validation that catches mistakes and asks the user to try so they do not get stuck

* Support for suggestions when multiple careers match the input rather than just returning one

* A knowledge base that is easy to extend. Adding a career just means adding a new fact and the rules do not need to change

None of these features required a lot of code.. Together they make the system feel more usable which I think matters even for a small project like this.

8. Implementation

The implementation is based on three things that Prolog's good at:

* Facts. These form the knowledge base, where each career is stored as a fact with its attributes

* Rules. These define the matching logic and check whether a career matches the users input

* Recursion. This handles the user-facing parts of the program like the menu and input prompts

I found it hard to get used to Prologs declarative style. In languages you write a sequence of instructions but in Prolog you describe relationships and constraints and the system figures out what follows. It took time to get the hang of it. There were times when I was writing Prolog code like it was Python, which does not work.

9. Challenges Faced

Prolog is less forgiving than languages I have used. The syntax rules are strict. If you make a mistake you get an error message that may not be very helpful. I had to be careful to catch errors like missing periods, lowercase variables and typos, in predicate names.

Input validation took longer than I expected. The basic idea is that when the user types something the system needs to reject it explain what the valid options are and ask again all without crashing. In Prolog this means writing a predicate that catches failure and tries again.

Getting the matching logic right required testing. The rule that suggests a career checks three conditions at and there are combinations of inputs that should return results and others that should not. Building a set of test cases and running through them manually took a while. It found a few edge cases where the logic was slightly off.

10. Learning Outcomes

I learned a lot from this project about how expert systems work. I mean I knew what an expert system was. It is different when you actually get to build one. I had to write the knowledge base and the rules. Then I got to see how Prologs inference engine worked. It was really interesting to watch it succeed and fail with inputs. When it failed I had to debug it which helped me understand how it works.

Prolog was a part of this project and I learned a lot from it. It is a way of programming and it is not like the other programming languages I have used. I had to build something with it. That

helped me understand how it works. I am not an expert in Prolog now. I know enough to read someone elses Prolog program and follow the logic.

I also learned about input validation, recursive control flow and how to build a rule-based inference system.. I got to translate a real-world decision process into formal logical rules, which was really interesting. It made me think about the assumptions I make when I am reasoning informally.

11. Future Improvements

The system works,. It is minimal. There are a things I could do to make it better:

* The career knowledge base is small many input combinations return no suggestions. I could add facts to the knowledge base, which would make it more useful.

* I could add an scoring system so the output is more useful. Now all matching careers are returned equally but if I could identify closer and looser matches that would be more informative.

* I could make an web-based interface so people who are not comfortable running programs in a terminal can use it. I would have to rebuild the end but the logic would stay the same.

* I could add input dimensions like salary range or industry sector which would make the suggestions more relevant to a specific user.

The Prolog architecture is good for extensions so I could add careers or input dimensions without having to redesign the whole system.

12. Conclusion

This project was a success I built a working career suggestion expert system in Prolog. It is not complex. It does what it says it does. Building it helped me understand the parts enough to debug them when they did not work.

What I found interesting was how much building something clarified concepts that had felt vague before. Expert systems, knowledge bases, inference rules, declarative programming. These are terms I had heard before. Now they mean something specific to me. I think that is the argument for project-based coursework it helps you understand things in a way that reading does not.

Prolog is a tool it is good for certain problems but not, for others.. Working with it was instructive because it forced me to think carefully about logical structure. I am glad I have that understanding even if I do not use Prolog much in the future.